

Enhancing Container Security with Per-Process Per-Container Egress Packet Filtering using eBPF

Ruturaj Mohite

International Institute of Information Technology Bangalore
Bengaluru, India
ruturaj.mohite@iiitb.ac.in

Balasubramanian Thangaraju

International Institute of Information Technology Bangalore
Bengaluru, India
b.thangaraju@iiitb.ac.in

Abstract—With growing use of containers and container orchestration in production systems, it is more important than ever to secure compromised containers from malicious uses. A compromised container could become a part of a botnet or could talk back to an attacker's server, exposing sensitive information. In this paper, we discuss the implementation of a packet filter which filters packets based on their L3 and L4 headers as well as their process and container of origin. To achieve this, we use extended Berkeley Packet Filter (eBPF) programs to probe into kernel functions and use hook points provided by the kernel to monitor and filter egress packets. Primarily, we use an eBPF program of type 'BPF_PROG_TYPE_CGROUP_SKB' that not only allows us to filter TCP/UDP packets, but also packets transmitted over raw sockets (e.g. ICMP packets). To attach this program to any new containers, we also explore an in-kernel solution to detect new containers at runtime. We conclude the paper by conducting an experiment to measure the latency overhead added by our proposed solution.

Keywords—eBPF, extended Berkeley packet filter, container security, kernel security, container networking

I. INTRODUCTION

Due to increasing use of containers and container orchestration in production systems, it is not only crucial to prevent the containers from being compromised, but to also think of solutions to mitigate the damage if containers are compromised. A compromised container can talk back to the attacker's server and be used as a part of a DDoS botnet, used to exfiltrate application data or be used to steal compute resources for cryptocurrency mining.

In this paper, we build a per-process per-container network filter to filter connections by containers to unwanted servers to ensure that network communication is allowed in a 'least privilege' way. To achieve this, we implement a pair of eBPF programs - one for filtering connections for a specific container, and another to detect new containers on the host system at runtime.

eBPF (extended Berkeley Packet Filter) is an in-kernel virtual machine that enables developers to write custom code hooked to certain points in the Linux kernel, allowing for real-time data processing and analysis in various applications, including networking, security, and performance monitoring. eBPF programs are of multiple types classified by their intended use cases and the points in the Linux kernel where they are attached.

BCC (BPF Compiler Collection) is a popular tool choice for writing eBPF programs. BCC is a collection of high level tools and libraries which ease the process of writing eBPF programs and save us from writing much of lower level eBPF code. Other than providing high level abstractions, BCC also provides a seamless user-kernel space interface with higher level languages such as Python [1]. BCC compiles eBPF programs on the fly using kernels headers installed on the target machine, hence providing some level of stability across different kernel versions.

II. BACKGROUND AND RELATED WORK

Use of eBPF for network monitoring and filtering has been widely explored. eBPF programs hooked to Express DataPath (XDP) [2], Traffic Control (TC) [3] subsystem and Linux Security Modules (LSMs) have been explored for traffic monitoring and filtering. Bertrone et al. (2018) [4] emulate the filtering behaviour of Linux's iptables using eBPF instead of netfilter hooks. Netflix [5] uses eBPF to monitor TCP flows using tracepoints. Cilium [6] uses eBPF to filter connections also based on the pod / container of origin.

While most of these works focus on network monitoring or generic packet filtering at the host or container level, Fournier's (2020) [7] paper discusses a solution to monitor and filter traffic at the process level.

Fournier (2020) [7], in their work, explores various approaches to filtering packets at a process and container level. Fournier's work details various methods and their shortcomings to achieve the desired granularity of filtering including Traffic Control eBPF programs, kprobes on kernel functions and kprobes on LSM functions. Ultimately, they use kprobes on the 'security_sk_classify_flow' LSM function. This method works very well for TCP / UDP packets, but fails to map packets using raw sockets (such as ICMP packets) back to their processes. This makes sense for ingress packet filtering, since ICMP requests are served by the kernel and not any user process. However, it is important to be able to filter these packets when they are sent out by a process to prevent attacks like ICMP tunnelling [8].

The implementation in this paper also fixes this problem by using a unique implementation approach and limiting our scope to outgoing traffic.

III. IMPLEMENTATION

Let's list our goals with this solution:

- Detect containers at run-time
- Monitor outgoing traffic from the containers
- Drop any packet not compliant with the user specified allow-list

It is also important to note that this filter must run on the host machine since our model assumes that the container is completely compromised.

The program is split into 3 components -

- eBPF program to filter egress connections ('filter').
- eBPF program to detect new containers at runtime ('if-probe').
- Userspace agent to listen for events and attach the eBPF programs ('agent').

A. Filtering connections

We use an eBPF program of type 'BPF_PROG_TYPE_CGROUP_SKB' to filter packets. This type of program is attached to a container's cgroup file descriptor and operates in the context of the socket buffer structure that contains information about network packets originating from the container. For egress traffic, the program is called in the 'ip_finish_output()' function in the kernel [9].

The ip_finish_output() function is responsible for fragmenting any packets that exceed MTU [10] in size and is one of the final hops for the outgoing packet before it's sent for layer 2 processing. 'ip_finish_output()' contains a call to the BPF_CGROUP_RUN_PROG_INET_EGRESS macro which calls the eBPF programs attached to the egress hook [11, Version 6.2, net/ipv4/ip_output.c, Line 313].

When this type of eBPF program returns a 0, the packet is dropped with reason 'SKB_DROP_REASON_BPF_CGROUP_EGRESS'. A non-zero return value results in '__ip_finish_output()' being called which performs the fragmentation and the packet passes through [11, Version 6.2, net/ipv4/ip_output.c, Line 340].

Our eBPF program takes the user defined specifications through BCC's convenient BPF_ARRAYs. Then matches the incoming packet's L3 and L4 header information with the specifications and decides whether to forward or drop the packet by returning the appropriate codes.

B. Mapping packets to processes

So far, we have figured out how to drop or forward packets originating from a container based on their L3 and L4 header information. We still need to find out a way to tell what process created the packet. The 'bpf_get_current_comm()' function can help us get the command of the process in the context of which the function is called. However, a 'BPF_PROG_TYPE_CGROUP_SKB' program does not run in process context, so the function cannot be called from this eBPF program.

To get around this limitation, we use another independent eBPF program that has access to the process context. We

choose an identifier field that both, this program and the filter program have access to. We then push these two (the identifier and the process command) as a key-value pair in a map shared between the two programs to access the process command from the filter program.

For this program, we opt for a kretprobe associated with the 'sock_alloc_file()' function. A kretprobe is a kernel feature that enables us to execute code each time a specified function is called, right before it finishes its execution, giving us access to the function's return value. The 'sock_alloc_file()' function is responsible for allocating and returning a 'file' structure that signifies a specific socket (even a raw socket) [13]. By attaching a kretprobe to this function we obtain the pointer to the 'file' structure that we use as the key for our map. The flow of the two programs is described in Fig. 1.

C. Detecting containers

As discussed earlier, a 'BPF_PROG_TYPE_CGROUP_SKB' program attaches to a cgroup file descriptor. Our next challenge is to detect any new containers that come up and obtain their cgroup file descriptors. This will help us automatically attach our filter program to any new containers.

Fournier (2020) [7] presents a way to detect containers at runtime using kprobes. Kprobes are similar to kretprobes, but instead of executing code at the end of the function call, they execute code at the beginning of the call, providing us access to the arguments passed to the function call. Using kprobes to certain kernel functions that are called when a container registers its veth as a device with the kernel can help us obtain the interface index (ifindex) of the veth's network interface. We can then obtain the container's ID using this ifindex and finally, the cgroup file descriptor's path can be obtained using the container's ID.

Primarily, we attach our kprobe to the 'register_netdevice()' function. The 'register_netdevice()' function takes a completed network device structure and adds it to the kernel interfaces [12] [13]. It is called in the network device context and is able to access the device's ifindex. A small detail to note is that for each container, this function is called twice - once when registering the container's veth and another time when registering the host's veth. We use a basic state machine in the kprobe to specifically capture container veth's registration as shown in Listing 1.

We also attach a similar kprobe to the 'unregister_netdevice_queue()' function to capture the event of a container stopping to detach the filter from the container.

After obtaining the ifindex, ifprobe pushes the ifindex and the event (container up or down) to a queue shared between this program and the userspace agent, the agent then compares it with ifindexes of the running containers to find the container ID of the new container.

D. Userspace programs

The jobs of the userspace program are to

- Ingest filter allow-list

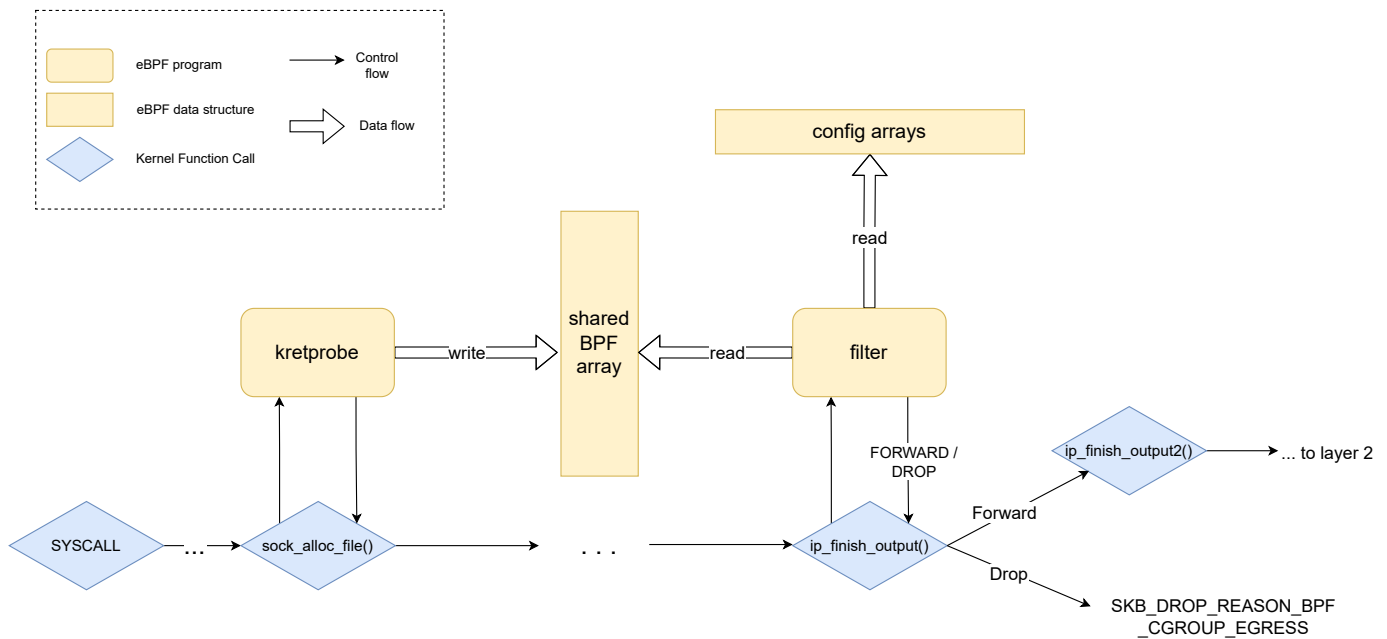


Fig. 1. Flow of the filter program with respect to kernel function calls. Kernel functions hook the attached eBPF code which reads from shared data structures.

```

switch (netdevice_ifindex->state) {
    // STATE_NEWLINK is the initial state
    // when the veths are newly created
    // When the function is called in
    // STATE_NEWLINK state, the container
    // device is being registered
    case STATE_NEWLINK:
        netdevice_ifindex->state
            = STATE_REGISTERING_PEER;

        netdevice_ifindex->dev = dev;
        break;
    // The container's device has been
    // registered if the state is
    // STATE_REGISTERING_PEER, now the
    // host's device is being registered.
    case STATE_REGISTERING_PEER:
        netdevice_ifindex->state
            = STATE_REGISTERING_HOST;
        break;
}

```

Listing 1: Handling state for netdevice registration

- Listen for events from the ifprobe program and attach the filter to containers
- Translate container ifindex to container ID

The userspace program takes the human readable allow-list and parses it to a set of BPF_ARRAYs that are shared between

```

[
  {
    "container_name": "curler",
    "policy": [
      {
        "process": "curl",
        "allow": [
          {
            "cidr4": "1.1.1.0/24",
            "ports": [80, 443]
          }
        ]
      }
    ]
  }
]

```

Listing 2: Allow-list JSON format (some trailing brackets are omitted for clarity)

the userspace program and the kernel-space BPF programs and are easy for the BPF programs to read. The user specifies their allow-list as a JSON in the format listed in the Listing 2:

When the ifprobe program pushes a new event, the userspace program reads the ifindex and event type (container up or container down) from the event structure. It then invokes a routine that finds the container ID from the ifindex.

To do this, the agent compares the ifindex with ifindexes of the running containers. The ifindexes of running containers can be obtained by the bash snippet in Listing 3.

Finally, it uses the container ID get the cgroup path for the container's cgroup and attaches / detaches the filter program to / from this cgroup.

```
# get pid of container from container ID
c_pid=$(docker inspect --format
'{{.State.Pid}}' "$container_id")

# symlink the network namespace file in
# /var/run/netns/NAME so it can be
# read by the `ip netns` command
ln -sf "/proc/${c_pid}/ns/net"
"/var/run/netns/ns-${c_pid}"

# use `ip` command to get the ifindex
ifindex=$(ip netns exec "ns-${c_pid}"
ip link show type veth)
```

Listing 3: Bash code to obtain ifindexes of a container given the container ID

```
/ # curl 1.1.1.1:3000
curl: (28) Failed to connect to 1.1.1.1
→ port 3000 after 129410 ms:
→ Operation timed out
```

Listing 4: attempting to curl a non-allow-listed service

The source code for the implementation is available at <https://github.com/gr455/contain>.

IV. RESULT

A. Function

TABLE I shows the action and outputs for the different relevant events that can occur within the container. The first column lists the relevant events, the second column describes the action taken by the programs in response and the third column describes the behaviour observed by the container processes. Listings of bash output also show the behaviour of the processes in the container when making allow-listed and non-allow-listed network requests. Listing 4 shows a HTTP request (using curl) to a non-allow-listed service for the 'curl' process on the container, which fails after repeated retransmission and timing out. Listing 5 shows an attempt to ping a non-allow-listed IP for the 'ping' process on the container which results in an EPERM (Permission error), while an allow-listed ping goes through as in Listing 6.

B. Performance Analysis

We conducted an experiment to measure the performance overhead of our implementation. The experiment was

```
/ # ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
ping: sendto: Operation not permitted
```

Listing 5: attempting to ping a non-allow-listed IP

```
/ # ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1): 56 data bytes
64 bytes from 1.1.1.1: seq=0 ttl=53
→ time=5.264 ms
64 bytes from 1.1.1.1: seq=1 ttl=53
→ time=14.970 ms
^C
--- 1.1.1.1 ping statistics ---
2 packets transmitted, 2 packets
→ received, 0% packet loss
round-trip min/avg/max =
→ 5.264/10.117/14.970 ms
```

Listing 6: attempting to ping an allow-listed IP

conducted by sending ping requests to an allow-listed IP (127.0.0.1) sequentially at 100ms intervals (to avoid queuing) and collecting their round-trip times for different number of filter rules set. The results of the experiment are shown in Fig 2. The choice to use ICMP packets sent to localhost was made for this experiment to keep the packets over the network lightweight, so that the overhead due to the filter is more pronounced.

The median latency went up by about $4\mu s$ on enabling the filter. Subsequently there was an insignificant increase in median latency ($\sim 1\mu s$) on increasing rule count from 20 to 100, which is an acceptable overhead. Scholz et al. (2018) [14] show that iptables and nftables drop in throughput more significantly with increase in rule count.

The device used for this experiment was equipped with Intel Core i7-8750H @ 2.2 GHz and 16 GB of RAM using Ubuntu 22.04.2 LTS (Linux 6.2.0-39-generic) as the operating system and running a container built with the alpine/curl image.

V. CONCLUSION

In this paper, we presented a solution to enhance container security through the implementation of a per-process, per-container packet filter for outgoing connections. The goal of this solution was to minimize impact by preventing containers from making malicious network connections in the event that they are compromised. We leveraged the power of eBPF, an in-kernel virtual machine, to develop two key programs: one for filtering outgoing connections and another for detecting new containers at runtime.

Our implementation overcomes challenges presented in related works, particularly in mapping outgoing ICMP packets and other raw socket traffic to their respective processes. By using a unique implementation approach and focusing on outgoing traffic, we achieved a more comprehensive solution.

Our results showcase the effectiveness of the implemented solution in filtering network traffic within containers, ensuring that only authorized connections are permitted, without adding much network overhead. The demonstration of the solution's behavior in various scenarios emphasizes its robustness.

TABLE I. Table detailing the behaviour of the filter for various possible events

Event	Action (What happens)	Effect (Behaviour within the container)
New container comes up	1. ifprobe pushes 'up' event to the shared event queue along with the container's veth's ifindex. 2. Userspace agent then obtains the container's name and ID to check against the policy. 3. If the container is in the policy, filter is attached to the container's cgroup	None
Container process makes request to an allow-listed service	1. Process' command name is obtained during socket file creation and pushed to a shared map. 2. filter program reads the packet's destination IP, port and process name to check against the policy. 3. Since the service is allow-listed, filter returns a zero exit code to let the packet pass.	The request is successful as expected.
Container process makes request to a non-allow-listed service	1. Process' command name is obtained during socket file creation and pushed to a shared map. 2. filter program reads the packet's destination IP, port and process name to check against the policy. 3. Since the service is disallowed, filter returns a non-zero exit code to drop the packet.	The request is unsuccessful. The packet is repeatedly retransmitted until timeout in case of TCP.
A container is stopped	1. ifprobe pushes a 'down' event to the shared event queue along with the container's veth's ifindex. 2. Userspace agent then obtains the container's ID. 3. filter, if attached, is detached from the container's cgroup.	None

VI. FUTURE WORK

B. CO-RE

Although the implementation described in this paper lays a foundation for advanced container security, there are a few considerations to be made before it's production-ready which were not explored in this paper.

A. Integration with Kubernetes

At this time, the daemon for this filter, which runs the ifprobe and attaches the filter to containers, has to be manually run on the target machine. This integration would aim to make it easy for engineers to deploy the filter on Kubernetes clusters, which are machine agnostic by design, by leveraging appropriate Kubernetes components like a DaemonSet.

BPF CO-RE (Compile Once – Run Everywhere) is a new approach for writing eBPF programs that do not require source code compilation on the target machine and can run across different kernel versions.

In this paper, we use BCC, which packages a compiler collection that compiles the BPF source code during runtime on the target host. Other than performance implications of compiling the source every time, the BCC approach relies on the right kernel headers being present on the target host. The structures and functions declared in these headers might change across kernel versions from what is expected in the eBPF programs. This makes our BPF code dependent on the kernel version of the target host.

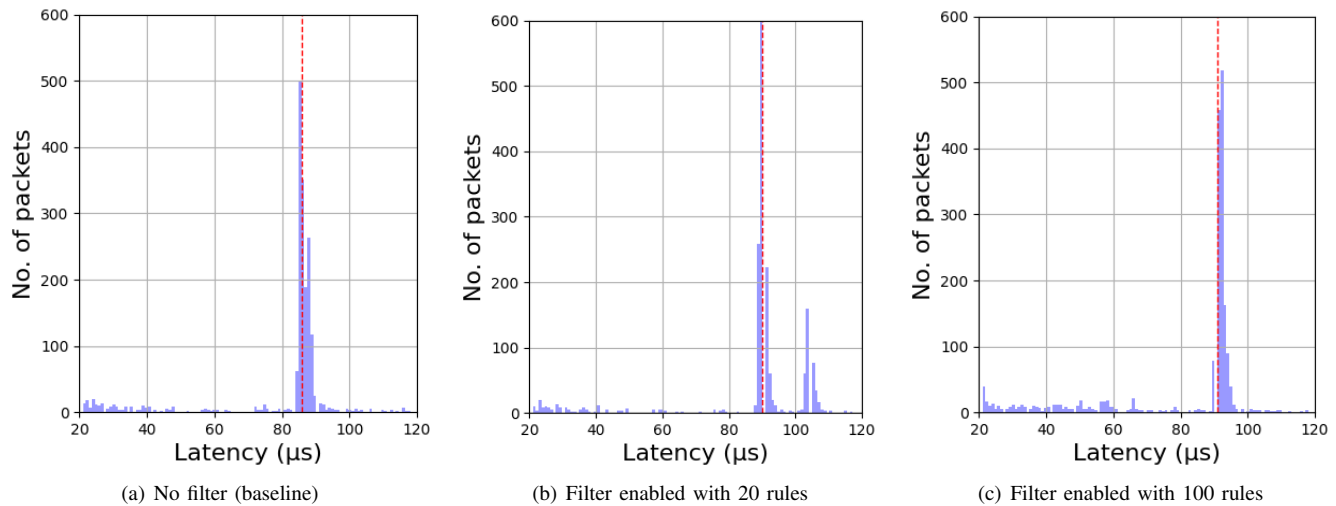


Fig. 2. Distribution of round-trip times for 2000 ICMP packets for increasing rule counts. Red dotted lines mark the medians of the distributions. The medians are a) $86\mu s$, b) $90\mu s$, c) $91\mu s$

REFERENCES

- [1] IO Visor Project. “iovisor/bcc,” GitHub, May 03, 2021. <https://github.com/iovisor/bcc>
- [2] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacífico, E. R. S. Santos, E. P. M. C. Júnior, and L. F. M. Vieira, “Fast Packet Processing with eBPF and XDP,” *ACM Computing Surveys*, vol. 53, no. 1, pp. 1–36, May 2020, doi: <https://doi.org/10.1145/3371038>.
- [3] T. Zavarella, “A methodology for using eBPF to efficiently monitor network behavior in Linux Kubernetes clusters,” Massachusetts Institute of Technology, 2022. Accessed: Jan. 14, 2024. [Online]. Available: <https://dspace.mit.edu/bitstream/handle/1721.1/145083/Zavarella-timzava-meng-eecs-2022-thesis.pdf>
- [4] M. Bertrone, S. Miano, J. Pi, F. Risso, and M. Tumolo, “Toward an eBPF-based clone of iptables,” in *The Technical Conference on Linux Networking*, The NetDev Society, Dec. 2018.
- [5] A. Tiagi, H. Ananthakrishnan, I. P. Carrero, and K. Lakshminarayan, “How Netflix uses eBPF flow logs at scale for network insight,” *Netflix Technology Blog*, Jun. 08, 2021. <https://netflixtechblog.com/how-netflix-uses-ebpf-flow-logs-at-scale-for-network-insight-e3ea997dca96> (accessed Jan. 14, 2024).
- [6] “Cilium Official Documentation,” docs.cilium.io. <https://docs.cilium.io> (accessed Jan. 14, 2024).
- [7] G. Fournier, “Process level network security monitoring and enforcement with eBPF,” in *Symposium sur la sécurité des technologies de l’information et des communications*, Jun. 2020.
- [8] A. Singh, O. Nordström, C. Lu, and A. L. M. dos Santos, “Malicious ICMP Tunneling: Defense against the Vulnerability,” *Information Security and Privacy*, pp. 226–236, 2003, doi: https://doi.org/10.1007/3-540-45067-x_20.
- [9] A. Maguire, “BPF: A Tour of Program Types,” *Oracle Linux Blog*, Jan. 08, 2019. <https://blogs.oracle.com/linux/post/bpf-a-tour-of-program-types> (accessed Jan. 14, 2024).
- [10] R. Serral and M. Gil, “A Linux networking study,” *ACM SIGOPS Operating Systems Review*, vol. 38, no. 3, pp. 1–11, Jul. 2004, doi: <https://doi.org/10.1145/1035834.1035835>.
- [11] Linus Torvalds (2023). *Linux [Operating system]*. Retrieved from <https://git.kernel.org/cgit/linux/kernel/git/torvalds/linux.git> (accessed January 16 2024)
- [12] A. Rubini, *Linux Device Drivers*. O’Reilly Media, 1998.
- [13] The Kernel Development Community (2023). *The Linux Kernel (Version 6.2) Documentation*. Retrieved from <https://www.kernel.org/doc/html/v6.2/networking/kapi.html> (accessed January 16, 2024)
- [14] D. Scholz, D. Raumer, P. Emmerich, A. Kurtz, K. Lesiak, and G. Carle, “Performance Implications of Packet Filtering with Linux eBPF,” 2018 30th International Teletraffic Congress (ITC 30), Sep. 2018, doi: <https://doi.org/10.1109/itc30.2018.00039>.